Students:
Juan Diego Castillo Amara [jdcastilloa1@gmail.com]
David Adrian Villanueva Guzmán [david.villanueva.guzman@gmail.com]

Lab1: ArUco Markers

Introduction

One common approach to simplify perception tasks is the use of fiducial markers, which provide easily detectable visual patterns with known geometry. Among these, ArUco markers are widely used due to their robustness, simplicity, and integration within the OpenCV library.

This laboratory session focuses on the generation, detection, and application of ArUco markers for different perception tasks. The work starts with the creation of individual markers and marker boards using predefined dictionaries. Then, real-time detection is implemented to identify markers in a video stream and overlay useful information such as their identifiers and contours.

After detection, the camera is calibrated using an ArUco board in order to obtain the intrinsic parameters and distortion coefficients. These parameters are later used to estimate the 3D pose of a marker with respect to the camera. Based on this pose estimation, several applications are developed, including augmented reality (drawing a virtual cube), relative pose estimation between two markers, and distance measurement.

The objective of this lab is to understand the complete pipeline from marker generation to 3D pose and distance estimation, while gaining practical experience with camera models, geometric transformations, and real-time computer vision using OpenCV.

Generation of ArUco Markers

ArUco markers are one clear example of fiducial markers. They consist of a graphical representation of a square that contains a bidimensional binary code determining its identifier. This square is a nxn matrix with black or white elements in each cell, creating a unique patter that can be identified and generated by OpenCV. ArUco markers are classified into dictionaries, each one containing a different group of M markers of N cells in size. Thus, to create a marker it is necessary to specify the dictionary, conformed by the group, the size in cells for each marker, the sub-group indicating the total possible markers in that group, and the id of the specific marker within that dictionary. For example, DICT_6X6_1000, accesses the dictionary of markers of 6x6 cells from a total of 1000 possible markers in that group even though there are $2\hat{3}6$ possible combinations with that size. The marker ID within that dictionary indicates the specific marker within that dictionary. Along with this information, it is necessary to indicate other parameters such as the size of the marker in pixels, whether to include a white contour to improve detection, the file name to save

the marker, the format, etc.

Generation of a single marker

The script `generate_marker.cpp` generates marker with the desired parameters. To use it, it expects 5 arguments with the following format:

```
$ ./generate_marker <dictionary> <id> <size> <file_name>
```

where:

1. dictionary (str): it consists of a string indicating the desired dictionary
2. id (int): the marker ID within the specified dictionary
3. size (int): the marker size in pixel
4. file_name (str): indicates the file name of the output image with the format included

An example of calling this script looks like this:

```
$ ./generate_marker DICT_6X6_1000 1 40 test.png
```

Which generates the following output image:

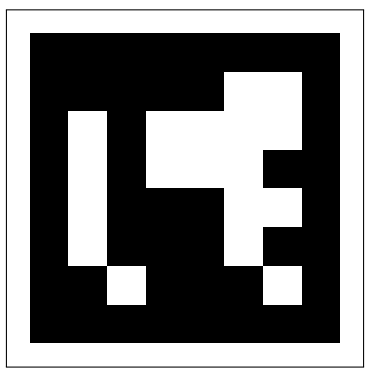

Figure 1: ArUco marker generated from DICT_6X6_1000, id 1, and a pixel size of 40.

Internally, the code extracts the arguments from the CLI and saves them as internal C++ variables. After that, it generates a marker using the built-in function generateImageMarker from the aruco library of OpenCV, as well as adding the white border to improve recognition in further exercises.

```
// Variables to store results
cv::Mat markerImage;
cv::Mat markerImage2;

// Generate the marker with the indicated parameters
cv::aruco::generateImageMarker(dictionary, markerId, markerSize, markerImage, 1);
```

```
// Add the white border
int left, right, top, bottom;
top = (int) (0.05*markerSize); bottom = top;
left = (int) (0.05*markerSize); right = left;

cv::copyMakeBorder(markerImage, markerImage2,
                   top, bottom, left, right,
                   cv::BORDER_CONSTANT, 255);
```

Generation of a an ArUco markers board

In this section once we were able to generate a single marker, we had to generate a board with multiple markers. The ArUco markers had to be different from each other.

The scrip **generate_board.cpp** generate the board with different markers. To use it, its required to specify 6 arguments with the following format:

```
$ ./generate_board <rows> <cols> <dictionary> <marker_size_px>
                   <separation_px> <file_name>
```

1. rows (int): the number of rows we want the board to have.
2. cols (int): the number of columns we want the board to have.
3. dictionary (str): its a string that indicates the desired dictionary of ArUco.
4. marker_size_px (int): the marker size in pixels.
5. separation_px (int): separation between the markers in pixels.
6. file_name (str): the name of the output image with the format included

The following image, shows a board generated calling the script like this:

```
$ ./generate_board 2 4 DICT_6X6_1000 50 20 board.png
```

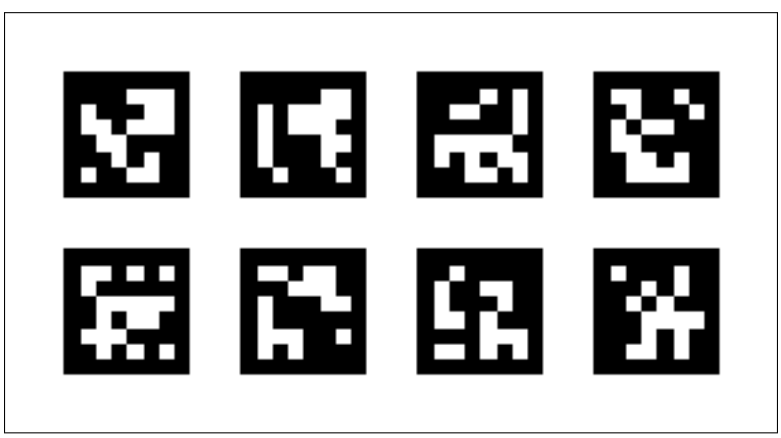

Figure 2: Example a board generated for calling /generate_board with the dictionary DICT_6X6_1000

The way the boar is generated is by using the method **GridBoard** from ArUco library of OpenCV. Since this method only generates the board without a white border we also generate a white image with the borders separated from the ArUco markes the separation distance that was specified. This in order to combined both images.

```
auto board = cv::makePtr<cv::aruco::GridBoard>(
    cv::Size(boardCols, boardRows),  // (cols, rows)
    markerSize,
    separation,
    dictionary
);

// Output image size
const int width = boardCols * markerSize + (boardCols + 1) * separation;
const int height = boardRows * markerSize + (boardRows + 1) * separation;

// Create the base image and the board over it
cv::Mat boardImage;
board->generateImage(cv::Size(width, height), boardImage, separation, 1);

// Save the file
std::stringstream ss;
//ss << "/home/david/Documents/IFRoS/Perception/labs/lab1/images/" << fileName;
ss << "/home/juan/Documents/IFROS/perception/LAB1Git/Lab1_Perception/images/" << fileName;
if (!cv::imwrite(ss.str(), boardImage)) {
    std::cerr << "Failed to write: " << ss.str() << "\n";
    return -1;
}
```

Detection of ArUco Markers

This section consists of implementing a program that recognizes ArUco markers from a specified dictionary in real time. The program will capture live video and output a window screen showing the frames captured. If any ArUco marker from the given dictionary is detected in the current frame, the program will overlay:

- A green frame for each marker
- A little red square in the upper left corner
- The identifier for each marker from the chosen dictionary

The script `detection.cpp` generates the output live-stream video and identifies the present markers from the given dictionary. To use it, it expects 3 arguments with the following format:

```
$ ./detection <video_source_no> <dictionary>
```

where:

1. video_source_no (int): Consists on the camera id from the device
2. dictionary (str): it consists of a string indicating the desired dictionary

An example of calling this script looks like this:

```
$ ./detection 0 DICT_6X6_1000
```

Which generates a video with this example frame:

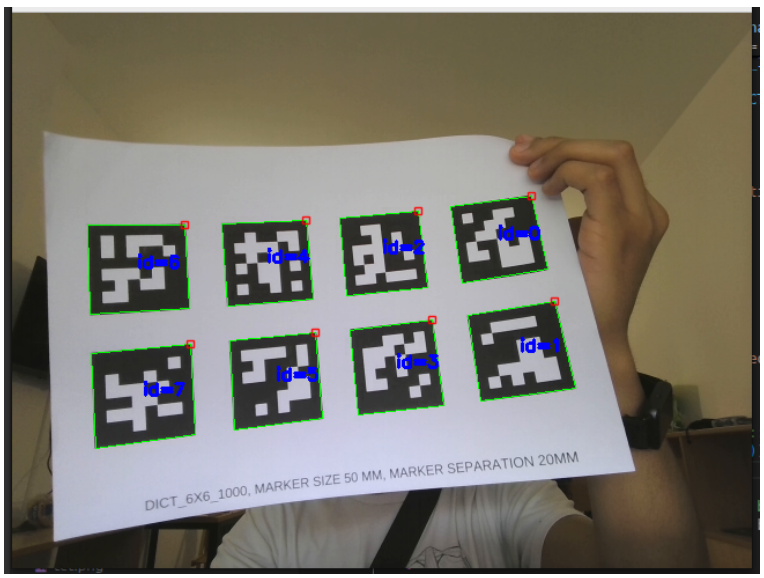

Figure 3: Example frame for calling /detection with the dictionary DICT_6X6_1000

ArUco markers are detected by the method `detectMarkers` from a detector, instance of the class `cv::aruco::ArucoDetector`. This method returns three arrays: the corners in the image plane for each detected marker, the id for each detected marker, and the candidates rejected by the detector. After detecting the markers, the function `cv::aruco::drawDetectedMarkers` receives the corners and the ids to draw the green contour, the id, and the red square for each detected marker. These functionalities are implemented with the code below inside the script called `detection.cpp`.

```
// For each frame in the video

// Create matrix to store the marker corners and ids
std::vector<int> markerIds;
std::vector<std::vector<cv::Point2f>> markerCorners, rejectedCandidates;

// Create the parametes for the detector and get the dictionary to use.
```

```
cv::aruco::DetectorParameters detectorParams = cv::aruco::DetectorParameters();
cv::aruco::Dictionary dictionary = cv::aruco::getPredefinedDictionary(dictionary_name);
cv::aruco::ArucoDetector detector(dictionary, detectorParams);

// Detect the markers on the image and draw the markers detected
detector.detectMarkers(imgOriginal, markerCorners, markerIds, rejectedCandidates);
cv::Mat outputImage = imgOriginal.clone();
cv::aruco::drawDetectedMarkers(outputImage, markerCorners, markerIds);
```

As seen in the script, the markers are drawn in an OutputImage as a copy of the original one, which includes the green contour, the id number, and the red square for each detected marker. Then this OutputImage is displayed with all the overlayed information.

Camera calibration using an ArUco board

This section focuses on calibrating the camera in order to estimate its intrinsic parameters and distortion coefficients using the previously generated ArUco board. The script `calibrate.cpp` performs the calibration process. To execute it, the following arguments must be provided:

```
/calibrate <video_source_no> <dictionary_name> <detector_parameters> <board_rows> \
           <board_cols> <marker_size> <separation> <file_name>
```

Where:

- video_source_no (int): Consist id the camera id.
- dictionary_name (str): It is a string indication the dictionary of the ArUco to detect.
- detector_parameters (str): Consist of a string
- board_rows (int): Consist of the amount of rows the board has.
- board_cols (int): Consist of the amount of columns the board has.
- marker_size (int): Consist of the size of the ArUco marker in meters.
- separation (int): Consist of the separation between the markes in meters.
- file_name (str): Consist of a string that says the name file of the calibration parametes

In order to achieve the calibration the program will be capturing a live video while detecting the ArUco markers and displaying the detection. Once the keyboard **"c"** is press it will save the image and it will store in a vector of all the markers corners, Ids detected and the size of the image at that specific moment. To be a good image for calibration is necessary that all ArUcos are detected.

```
// Detect the markers on the imageand draw the markers detected
    detector.detectMarkers(imgOriginal, markerCorners, markerIds, rejectedCandidates);
    detector.refineDetectedMarkers(imgOriginal, gridboard, markerCorners,
                                   markerIds, rejectedCandidates);
```

```
    cv::Mat outputImage = imgOriginal.clone();
    cv::aruco::drawDetectedMarkers(outputImage, markerCorners, markerIds);
    cv::imshow("imgOriginal", outputImage);

    // Gets the key pressed
    charCheckForKey = cv::waitKey(1);

    // Capture image when the c key is pressed and save it
    if (charCheckForKey == 'c' && !markerIds.empty()){

        // Save the image
        printf("Capturing image %d\n", nframe);
        std::string fileName = \
            "/home/david/Documents/IFRoS/Perception/labs/lab1/images/image_" + \
            std::to_string(nframe) + ".jpg";

        // Save data for calibration
        allMarkerCorners.push_back(markerCorners);
        allMarkerIds.push_back(markerIds);
        imageSize = imgOriginal.size();

        // cv::imwrite(fileName, imgOriginal);
        nframe++;
    }
}
```

We tested two different methods to calibrate the camera. The fist method we use is by using the **calibrateCameraAruco** method form ArUco library. In order to use this method we had to prepare data into a vector that had all the markers corners, ids and markers count per frame. Then with the information that was specified in the headers, we create a board using the method **GridBoard**, to use this board as the ground truth for the calibration.

```
// --- Calibration process ---
cv::Mat cameraMatrix, distCoeffs;

// if(calibrationFlags & CALIB_FIX_ASPECT_RATIO) {
//     cameraMatrix = cv::Mat::eye(3, 3, CV_64F);
//     cameraMatrix.at<double>(0, 0) = aspectRatio;
// }

// Prepare data for calibration
size_t nFrames = allMarkerCorners.size();
std::vector<cv::Mat> rvecs, tvecs;
```

```
double repError;

bool calibrationType = true; // true for aruco, false for classic
if (calibrationType == true){ // Calibration using calibrateCameraAruco
    // Build markerCounterPerFrame
    std::vector<std::vector<cv::Point2f>> allCorners;
    std::vector<int> allIds;
    std::vector<int> markerCounterPerFrame;

    // Store all concatenated corners and ids, and the number of markers per frame
    for (size_t frame = 0; frame < nFrames; ++frame) {
        const auto& ids = allMarkerIds[frame];
        const auto& corners = allMarkerCorners[frame];
        markerCounterPerFrame.push_back((int)ids.size());
        for (size_t i = 0; i < ids.size(); ++i) {
            allCorners.push_back(corners[i]);   // marker i corners
            allIds.push_back(ids[i]);           // marker i id
        }
    }

    auto board = cv::makePtr<cv::aruco::GridBoard>(
        cv::Size(boardRows, boardCols),
        markerSize,
        separation,
        dictionary
    );

    repError = cv::aruco::calibrateCameraAruco(
        allCorners,          // per-frame corners
        allIds,              // per-frame ids
        markerCounterPerFrame,     // markers per frame
        board,                     // known board geometry
        imageSize,                 // image size
        cameraMatrix,              // output
        distCoeffs,                // output
        rvecs, tvecs, 0
    );
}
```

The second method was based on the generic `cv::calibrateCamera()` function from OpenCV. In this case, an additional step is required before calibration. The detected image points must be matched with their corresponding 3D object points using the function `matchImagePoints()`. This matching process is repeated for all captured images. Unlike `calibrateCameraAruco()`, which internally performs the correspondence association, the generic calibration function requires this

step to be done explicitly.

```
else{ // Calibration using cameraCalibrate
    std::vector<cv::Mat> processedObjectPoints, processedImagePoints;

    for(size_t frame = 0; frame < nFrames; frame++) {
        cv::Mat currentImgPoints, currentObjPoints;

        gridboard.matchImagePoints(allMarkerCorners[frame],
        allMarkerIds[frame], currentObjPoints, currentImgPoints);

        if(currentImgPoints.total() > 0 && currentObjPoints.total() > 0) {
            processedImagePoints.push_back(currentImgPoints);
            processedObjectPoints.push_back(currentObjPoints);
        }
    }

    // Calibrate camera
    repError = cv::calibrateCamera(processedObjectPoints, processedImagePoints,
                                   imageSize, cameraMatrix, distCoeffs,
                                   cv::noArray(), cv::noArray(), cv::noArray(),
                                   cv::noArray(), cv::noArray()
                                );
}
```

The results are then saved into a yaml file containing the calibration parameters (intrinsic and distortion coefficients) from the camera used. This was done through the following section of code, which also prints out the values:

```
// Prints the outputs and saves the file
std::cout << "Reprojection error: " << repError << "\n";
std::cout << "Camera matrix:\n" << cameraMatrix << "\n";
std::cout << "Dist coeffs:\n" << distCoeffs << "\n";

// Save to YAML
cv::FileStorage fs(fileName, cv::FileStorage::WRITE);
fs << "camera_matrix" << cameraMatrix;
fs << "dist_coeffs" << distCoeffs;
fs << "reprojection_error" << repError;
fs.release();
```

After performing the calibration process using the ArUco board, the intrinsic camera matrix and distortion coefficients were obtained. The reprojection error was also computed to evaluate the calibration accuracy.

The intrinsic camera matrix $\mathbf{K}$ is given by:

$$\mathbf{K} = \begin{bmatrix} 412.10 & 0 & 320.89 \\ 0 & 412.43 & 227.83 \\ 0 & 0 & 1 \end{bmatrix} \tag{1}$$

From this matrix:

- $f_x = 412.10$ px
- $f_y = 412.43$ px
- $c_x = 320.89$ px
- $c_y = 227.83$ px

The distortion coefficients are:

$$\mathbf{d} = \begin{bmatrix} 0.12885 & -0.27831 & -0.01400 & 0.00059 & 0.18664 \end{bmatrix} \tag{2}$$

where the coefficients correspond to:

$$(k_1, k_2, p_1, p_2, k_3)$$

The final reprojection error obtained after calibration was:

$$\text{Reprojection Error} = 0.7784 \text{ pixels} \tag{3}$$

This value indicates a reasonably good calibration, since the average reprojection error is below one pixel.

Using ArUco Markers for Augmented Reality and distance measurements

One of the main applications of ArUco markers as fiducial markers is their use in Augmented Reality (AR). There are two main ways to achieve this result: either by using fiducial markers or artificial intelligence to recognize objects in the scene.

Pose estimation

The simplest application, as well as the basis for AR, is 3D pose estimation of objects of, more specifically, ArUco markers. Given the intrinsic parameters for the camera (camera calibration), it is possible to find the 3D pose from 3D-2D correspondances.
This is solved in the script `pose_estimation.cpp` which is called by the following heading:

```
$ ./pose_estimation <video_source_no> <dictionary> <id> <marker_size>
```

where:

- video_source_no (int): Consists on the camera id from the device.
- dictionary (str): it consists of a string indicating the desired dictionary.
- marker_size (float): The real size of the marker in metric units, must be greater than 0.

Below is an example of calling this function and its result:

```
$ ./pose_estimation 0 DICT_4X4_50 0 50
```

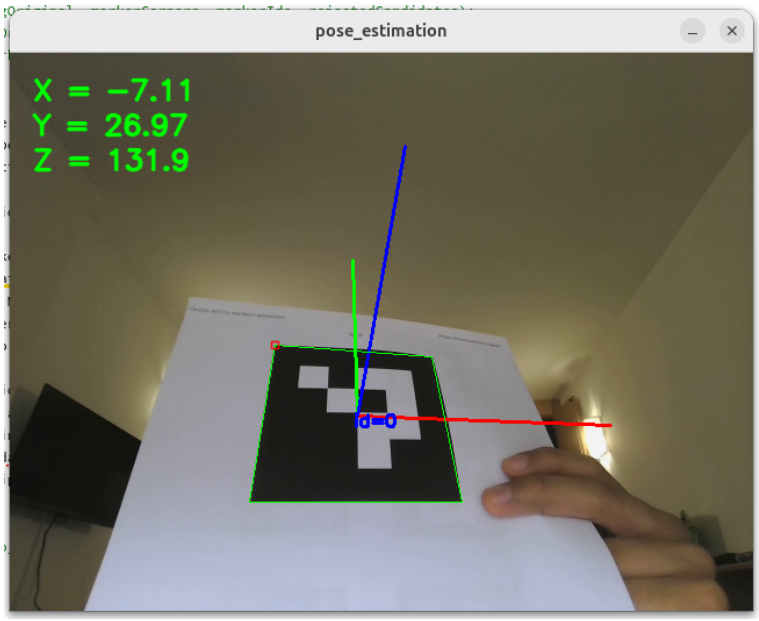

Figure 4: Example frame for calling /pose_estimation with the dictionary DICT_4X4_50

In order to achieve this result, first it is necessary to obtain the camera parameters from the yaml file, obtained from section Camera Calibration. This is done through the function called `readCameraParams` which outputs the desired camera parameters.

```
bool readCameraParams(
    cv::Mat& camMatrix,
    cv::Mat& distCoeffs){

    // Extract the camera parameters file name
    std::string filename = "../src/calibration_v2.yaml";

    if (filename.empty()) {
        std::cerr << "Camera parameters file not provided." << std::endl;
        return false;
}

    cv::FileStorage fs(filename, cv::FileStorage::READ);
    if (!fs.isOpened()) {
        std::cerr << "Cannot open camera file: " << filename << std::endl;
        return false;
    }
```

```
    // Save the parameters from the file
    fs["camera_matrix"] >> camMatrix;
    fs["dist_coeffs"] >> distCoeffs;

    if (camMatrix.empty() || distCoeffs.empty()) {
        std::cerr << "Invalid camera parameters in file." << std::endl;
        return false;
    }

    // Return true if success
    return true;
}
```

After that, markers are detected in each frame of the video, but only processing the marker id of interest. The core function is called `cv::solvePnP()`, which finds an object pose from 3D-2D point correspondences. This function returns the rotation and the translation vectors that transform a 3D point expressed in the object coordinate frame to the camera coordinate frame, using different methods, by default SOLVEPNP_P3P. The function expects 4 main arguments: the known 3D points (the marker corners in its own coordinate system), the observed 2D projections (the marker corners obtained by the detector), the camera parameters (intrinsics and distortion coefficients), and finally the variables to store the translation and rotation vectors to convert the observed 2D points into the given 3D points.

With these transformations, it is possible to draw the green contour around the marker, the little red square in one of the corners, as well as the frame axes in the center. This center value is also printed on the image.

The known 3D points, which correspond to the marker corner in the marker's coordinate system, are defined as:

```
// set coordinate system
cv::Mat objPoints(4, 1, CV_32FC3);
objPoints.ptr<cv::Vec3f>(0)[0] = cv::Vec3f(-markerSize/2.f, markerSize/2.f, 0);
objPoints.ptr<cv::Vec3f>(0)[1] = cv::Vec3f(markerSize/2.f, markerSize/2.f, 0);
objPoints.ptr<cv::Vec3f>(0)[2] = cv::Vec3f(markerSize/2.f, -markerSize/2.f, 0);
objPoints.ptr<cv::Vec3f>(0)[3] = cv::Vec3f(-markerSize/2.f, -markerSize/2.f, 0);
```

which consist on the corner points measured from the center of the marker in the real-world coordinates given when calling the script. Furthermore, when detecting markers in each video frame and ensuring the id of interest is present, the transformations are calculated and the image is overlayed with the important information.

```
// In each frame, check whether any markers were detected
if (!markerIds.empty()) {
```

```
    // Iterate over all the markers
    for (size_t i = 0; i < nMarkers; ++i) {

        // Skip markers that do not match the id of interest
        if(markerIds[i] != markerId) { continue; }

        // Estimate pose of the marker
        cv::solvePnP(objPoints, markerCorners.at(i), camMatrix,
                    distCoeffs, rvecs.at(i), tvecs.at(i));

        // Draw the axis for the marker
        cv::drawFrameAxes(outputImage, camMatrix, distCoeffs,
                        rvecs[i], tvecs[i], markerSize * 1.5f, 2);

        // Add a border to the marker detected
        std::vector<std::vector<cv::Point2f>> oneCorners{ markerCorners[i] };
        std::vector<int> oneIds{ markerIds[i] };
        cv::aruco::drawDetectedMarkers(outputImage, oneCorners, oneIds);

        // Put coordinates in the image
        // XYZ text
        cv::Vec3d tvec = tvecs[i];
        char bufX[10], bufY[10], bufZ[10];
        std::snprintf(bufX, sizeof(bufX), "X = %.5f", tvec[0]);
        std::snprintf(bufY, sizeof(bufY), "Y = %.5f", tvec[1]);
        std::snprintf(bufZ, sizeof(bufZ), "Z = %.5f", tvec[2]);

        cv::putText(outputImage, bufX, cv::Point(20,40),
        cv::FONT_HERSHEY_SIMPLEX, 0.8, cv::Scalar(0,255,0), 2, cv::LINE_AA);

        cv::putText(outputImage, bufY, cv::Point(20,70),
        cv::FONT_HERSHEY_SIMPLEX, 0.8, cv::Scalar(0,255,0), 2, cv::LINE_AA);

        cv::putText(outputImage, bufZ, cv::Point(20,100),
        cv::FONT_HERSHEY_SIMPLEX, 0.8, cv::Scalar(0,255,0), 2, cv::LINE_AA);

        //
        End the for loop when the marker of interest is processed
        break;
    }
}
```

Drawing a cube

In order to build a cube above the ArUco marker is necessary to estimate the 3D pose of this. The script **pose_estimation** use the pose estimated of the marker and estimates the position of the cube's vertices. This allows the cube follow the marker position and change its form if the marker changes. This script is called by the following header:

```
./generate_marker <camera_id > <dictionary> <id> <size>
```

Where:

- camera_id (int): Consist on the camera id.
- dictionary (str): It is a string indication the dictionary of the ArUco to detect.
- id (int): Consist of the ArUco i from the directory in which the cube wants to be drawn.
- size (int): Consist on the marker's size in meters.

Below is an example of calling this function and its result:

In order to achieve this result, the process is similar to the one used in **pose_estimation**. In this case after estimating the pose of the marker, it is possible to start estimating the axis positions on the image. The function **drawCube** was created in order to estimate all the axis depending if the pose of the marker changed over time. Once the axis were estimated this once wer dawned on the displayed image.

```
void drawCube(
    // Creates the 3D points of the cube and projects them to the image plane,
    // then draws the edges of the cube
    cv::InputOutputArray image, cv::InputArray camera_matrix,
    cv::InputArray dist_coeffs, cv::InputArray rvec, cv::InputArray tvec,
    float l){
    float half_l = l / 2.0;

    // Project cube points
    std::vector<cv::Point3f> axis_points;
    axis_points.push_back(cv::Point3f(half_l, half_l, l));
    axis_points.push_back(cv::Point3f(half_l, -half_l, l));
    axis_points.push_back(cv::Point3f(-half_l, -half_l, l));
    axis_points.push_back(cv::Point3f(-half_l, half_l, l));
    axis_points.push_back(cv::Point3f(half_l, half_l, 0));
    axis_points.push_back(cv::Point3f(half_l, -half_l, 0));
    axis_points.push_back(cv::Point3f(-half_l, -half_l, 0));
    axis_points.push_back(cv::Point3f(-half_l, half_l, 0));

    std::vector<cv::Point2f> image_points;
    projectPoints(axis_points, rvec, tvec, camera_matrix, dist_coeffs, image_points);
```

```
    // Draw cube edges lines
    cv::line(image, image_points[0], image_points[1], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[0], image_points[3], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[0], image_points[4], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[1], image_points[2], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[1], image_points[5], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[2], image_points[3], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[2], image_points[6], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[3], image_points[7], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[4], image_points[5], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[4], image_points[7], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[5], image_points[6], cv::Scalar(255, 0, 0), 3);
    cv::line(image, image_points[6], image_points[7], cv::Scalar(255, 0, 0), 3);
}
```

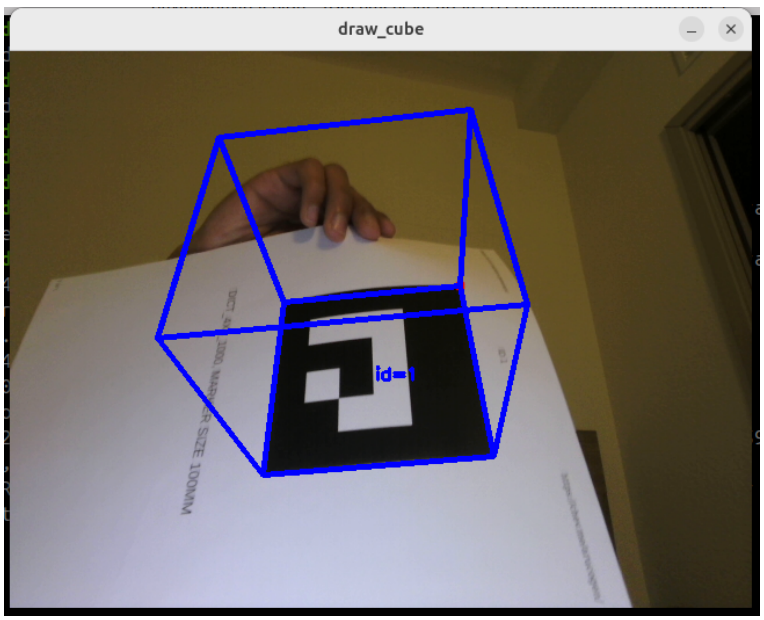

Figure 5: Example frame for calling /draw_cube with the dictionary DICT_6X6_1000

Knowing the 3D pose of an object given an ArUco marker, it is possible to obtain the same information from several markers and calculate the relative position of one marker with respect to the other. In this case, the script `relative_pose_estimation.cpp` estimates the relative position from the right-most marker relative to the left-most marker from a given set of 2 markers. The script is called by the following header:

```
$ ./relative_pose_estimation <camera_id > <dictionary> <id1> <id2> <size>
```

where:

- video_source_no (int): Consists on the camera id from the device.

- dictionary (str): it consists of a string indicating the desired dictionary.
- id1 and id2 (int): ID of the markers of interest from the indicated dictionary.
- marker_size (float): The real size of the marker in metric units, must be greater than 0.

Below is an example of calling this function and one output frame of the generated video:

```
$ ./relative_pose_estimation 0 DICT_6X6_1000 0 1 50
```

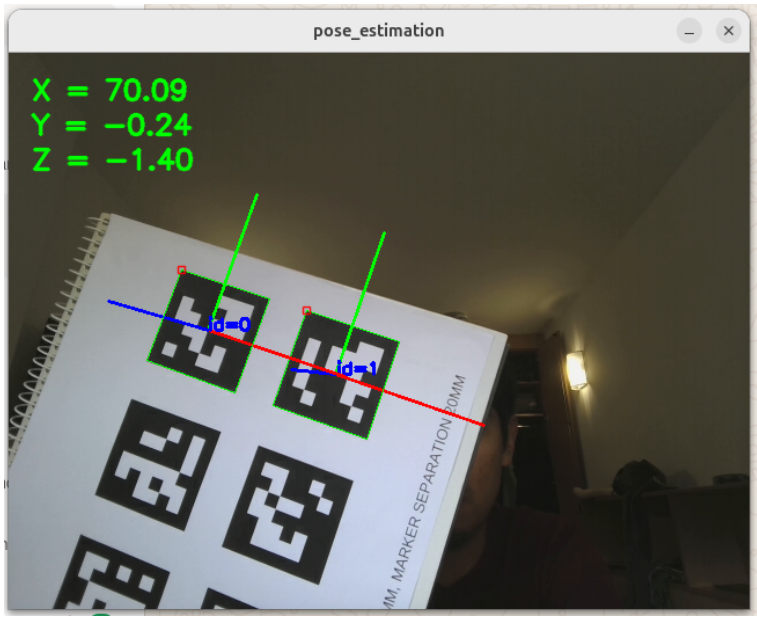

Figure 6: Example frame for calling /relative_pose_estimation with the dictionary DICT_6X6_1000

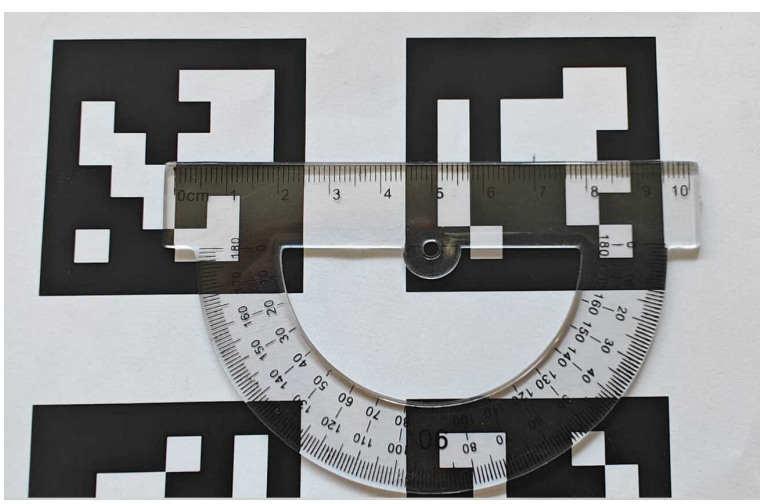

Figure 7: Real distance measurement from the example

As seen in the image, the relative position of ArUco 1 relative to ArUco 0 is marked as (70.09, -0.24, -1.4) mm. Image 7 shows the real distance and position from one ArUco to the other, where in this case the relative displacement is 70 mm in X, while Y and Z remain 0. These values are obtained with good precision by the camera, and the slight errors (specially in the Y and Z axis) might be due to errors in calibration or small imperfections in the sheet, pictures taken to calibrate, lightning, etc.

The logic implemented in `relative_pose_estimation.cpp` is similar to the one used in `pose_estimation.` but this time, the goal is to identify two ID's instead of only one. Therefore, in each frame, it is checked whether the two markers of interest have been detected. If so, their transformations are detected and their axes and border are drawn on top of them. This proceadure is shown in the code below:

```
std::vector<cv::Vec3d> rvecs(2), tvecs(2);
int counter = 0;

if (!markerIds.empty()) {
    // Calculate pose for each marker
    for (size_t i = 0; i < nMarkers; ++i) {
        if(markerIds[i] == markerId1 || markerIds[i] == markerId2) {

            // Store the marker corners of interest
            // interest_markerCorners.push_back(markerCorners[i]);

            // Estimate pose of the marker
            cv::solvePnP(objPoints, markerCorners.at(i), camMatrix,
            distCoeffs, rvecs.at(counter), tvecs.at(counter),
            false, cv::SOLVEPNP_IPPE_SQUARE);

            // Draw the axis for the marker
            cv::drawFrameAxes(outputImage, camMatrix, distCoeffs,
            rvecs[counter], tvecs[counter], markerSize * 1.5f, 2);

            // Add a border to the marker detected
            std::vector<std::vector<cv::Point2f>> oneCorners{ markerCorners[i] };
            std::vector<int> oneIds{ markerIds[i] };
            cv::aruco::drawDetectedMarkers(outputImage, oneCorners, oneIds);

            counter++;

            // Avoid unnecessary calculations
            if (counter == 2) { break; }
        }
    }
```

```
    ...
}
```

For each detected marker, the pose estimation provides a rotation vector $\mathbf{r}$ and a translation vector $\mathbf{t}$ such that:

$$\mathbf{X}_{cam} = \mathbf{R}\,\mathbf{X}_{marker} + \mathbf{t} \tag{4}$$

where:

- $\mathbf{X}_{marker}$ is a 3D point expressed in the marker coordinate frame,
- $\mathbf{X}_{cam}$ is the same point expressed in the camera coordinate frame,
- $\mathbf{R}$ is the rotation matrix obtained from $\mathbf{r}$ via `cv::Rodrigues` function,
- $\mathbf{t}$ is the translation vector.

This transformation can be written in homogeneous form as:

$$^{cam}\mathbf{T}_m = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{5}$$

Let:

$$^{cam}\mathbf{T}_{m_1} = \begin{bmatrix} \mathbf{R}_1 & \mathbf{t}_1 \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{6}$$

$$^{cam}\mathbf{T}_{m_2} = \begin{bmatrix} \mathbf{R}_2 & \mathbf{t}_2 \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{7}$$

These describe the poses of marker 1 (left marker) and marker 2 (right marker) with respect to the camera frame.

The pose of marker 2 expressed in the coordinate frame of marker 1 is:

$$^{m_1}\mathbf{T}_{m_2} = {}^{m_1}\mathbf{T}_{cam}\,{}^{cam}\mathbf{T}_{m_2} \tag{8}$$

Since:

$$^{m_1}\mathbf{T}_{cam} = ({}^{cam}\mathbf{T}_{m_1})^{-1} = \begin{bmatrix} \mathbf{R}_1^T & -\mathbf{R}_1^T\mathbf{t}_1 \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{9}$$

it is possible to obtain:

$$^{m_1}\mathbf{T}_{m_2} = \begin{bmatrix} \mathbf{R}_1^T\mathbf{R}_2 & \mathbf{R}_1^T(\mathbf{t}_2 - \mathbf{t}_1) \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{10}$$

The relative position of marker 2 expressed in marker 1's coordinate system is therefore:

$$\mathbf{t}_{rel} = \mathbf{R}_1^T\,(\mathbf{t}_2 - \mathbf{t}_1) \tag{11}$$

where:

- $\mathbf{t}_1$ and $\mathbf{t}_2$ are the translation vectors of markers 1 and 2 in the camera frame,
- $\mathbf{R}_1$ is the rotation matrix of marker 1.

To determine which marker is located at the left or right in the image, the 3D center of each marker is projected onto the image plane.

The center of each marker in its own coordinate frame is:

$$\mathbf{p}_{m_i} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{12}$$

Using the estimated pose $(\mathbf{R}_i, \mathbf{t}_i)$ and the camera intrinsic parameters, the center is projected into the image plane, resulting in pixel coordinates:

$$\mathbf{p}_i^{img} = \begin{bmatrix} u_i \\ v_i \end{bmatrix} \tag{13}$$

Since the center point is the origin of the marker frame, its position in the camera frame is simply:

$$\mathbf{X}_{cam,i} = \mathbf{t}_i \tag{14}$$

The horizontal pixel coordinates u_1 and u_2 of the two projected centers are then compared. The left/right relationship is defined as:

$$u_2 > u_1 \quad \Rightarrow \quad \text{Marker 2 is to the right of Marker 1} \tag{15}$$

If $u_2 > u_1$, marker 1 is selected as the reference frame and the relative translation is computed as:

$$\mathbf{t}_{rel} = \mathbf{R}_1^T (\mathbf{t}_2 - \mathbf{t}_1) \tag{16}$$

Otherwise, marker 2 is selected as the reference frame and:

$$\mathbf{t}_{rel} = \mathbf{R}_2^T (\mathbf{t}_1 - \mathbf{t}_2) \tag{17}$$

This ensures that the relative pose is always expressed in the coordinate frame of the marker that appears left-most in the image. This logic is implemented right after both markers are detected in each frame:

```
if (counter == 2){

    // Transforms rotation vectors into rotation matrices
    cv::Mat R1, R2;
    cv::Rodrigues(rvecs[0], R1);
    cv::Rodrigues(rvecs[1], R2);

    // Get the translation vectors of the two markersstd::
    cv::Mat t1 = (cv::Mat_<double>(3,1) << tvecs[0][0], tvecs[0][1], tvecs[0][2]);
    cv::Mat t2 = (cv::Mat_<double>(3,1) << tvecs[1][0], tvecs[1][1], tvecs[1][2]);

    // Deinfe the 3D points relative to the markers' coordinate systems
    // marker-frame center for marker 1
    std::vector<cv::Point3f> Pm1 = { cv::Point3f(0.f, 0.f, 0.f) };
```

```
// marker-frame center for marker 2
std::vector<cv::Point3f> Pm2 = { cv::Point3f(0.f, 0.f, 0.f) };

// projected marker centers into the image plane
std::vector<cv::Point2f> pim1, pim2;

// Project the marker centers into the image plane
cv::projectPoints(Pm1, rvecs[0], tvecs[0], camMatrix, distCoeffs, pim1);
cv::projectPoints(Pm2, rvecs[1], tvecs[1], camMatrix, distCoeffs, pim2);

// U coordinates of both centers
double u1 = pim1[0].x;
double u2 = pim2[0].x;

// translation of marker 2 expressed in marker 1 coordinates
cv::Mat t_rel;
if (u2 > u1) {
    t_rel = R1.t() * (t2 - t1);
}
else {
    t_rel = R2.t() * (t1 - t2);
}

// Get relative translation in x, y and z
double delta_x = t_rel.at<double>(0);
double delta_y = t_rel.at<double>(1);
double delta_z = t_rel.at<double>(2);

// Assign the text to show and format it
char bufX[10], bufY[10], bufZ[10];
std::snprintf(bufX, sizeof(bufX), "X = %.5f", delta_x);
std::snprintf(bufY, sizeof(bufY), "Y = %.5f", delta_y);
std::snprintf(bufZ, sizeof(bufZ), "Z = %.5f", delta_z);

// Show the text in the image
cv::putText(outputImage, bufX, cv::Point(20,40),
cv::FONT_HERSHEY_SIMPLEX, 0.8, cv::Scalar(0,255,0), 2, cv::LINE_AA);

cv::putText(outputImage, bufY, cv::Point(20,70),
cv::FONT_HERSHEY_SIMPLEX, 0.8, cv::Scalar(0,255,0), 2, cv::LINE_AA);

cv::putText(outputImage, bufZ, cv::Point(20,100),
cv::FONT_HERSHEY_SIMPLEX, 0.8, cv::Scalar(0,255,0), 2, cv::LINE_AA);
```

```
}
```

Distance estimation

Once the pose of two markers is estimated with respect to the camera, it is possible to compute the Euclidean distance between them. In this implementation, the translation vectors obtained from `cv::solvePnP()` are directly used. Each translation vector $\mathbf{t}_i = (t_{x_i}, t_{y_i}, t_{z_i})$ represents the position of marker i expressed in the camera reference frame.

If both markers are detected in the same frame, their relative displacement in the camera coordinate system can be computed as:

$$\Delta x = t_{x_2} - t_{x_1}, \quad \Delta y = t_{y_2} - t_{y_1}, \quad \Delta z = t_{z_2} - t_{z_1} \tag{18}$$

The 3D distance between the two markers is then calculated using the Euclidean norm:

$$D = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} \tag{19}$$

This value represents the straight-line distance between the centers of both markers in metric units (millimeters in our case).

To improve visualization, the origin of each marker (which corresponds to its center) is projected into the image plane using `cv::projectPoints()`, and a green line is drawn between both projected points. The computed distance is also displayed on the image in real time.

The obtained results show a good agreement between the measured distance and the real distance measured manually (see Figure 8). Small differences can be observed, mainly due to calibration inaccuracies, noise in corner detection, slight marker bending, and illumination conditions. Even with these limitations, the estimation provides a reliable approximation suitable for robotic perception tasks.

The script `distance_estimation.cpp` is called with the following heading:

```
$ ./relative_pose_estimation <camera_id > <dictionary> <id1> <id2> <size>
```

where:

- camera_id (int): represents the desired camera index
- dictionary (str): it consists of a string indicating the desired dictionary.
- id1, id2 (int): represent the desired marker ID's to find to estimate its relative distance
- marker_size (float): The real size of the marker in metric units, must be greater than 0.

```
// If all markers have been detected, we can calculate the relative
// pose and draw the line between the two markers
if (counter == 2){
    // Calculate the relative translation between the two markers
    double delta_x = tvecs[1][0] - tvecs[0][0];
    double delta_y = tvecs[1][1] - tvecs[0][1];
```

```
    double delta_z = tvecs[1][2] - tvecs[0][2];

    std::vector<cv::Point3f> origin(1, cv::Point3f(0,0,0));
    std::vector<cv::Point2f> imgPt0, imgPt1;

    // Project origin of marker 1
    cv::projectPoints(origin,
                      rvecs[0],
                      tvecs[0],
                      camMatrix,
                      distCoeffs,
                      imgPt0);

    // Project origin of marker 2
    cv::projectPoints(origin,
                      rvecs[1],
                      tvecs[1],
                      camMatrix,
                      distCoeffs,
                      imgPt1);

    cv::line(outputImage, imgPt0[0], imgPt1[0], cv::Scalar(0, 255, 0), 2);

    // Assign the text to show and format it
    double distance_val = sqrt(delta_x*delta_x + delta_y*delta_y + delta_z*delta_z);
    char distance_str[15];
    std::snprintf(distance_str, sizeof(distance_str), "D = %.0f mm", distance_val);

    // Show the text in the image
    cv::putText(outputImage, distance_str, cv::Point(20,40),
    cv::FONT_HERSHEY_SIMPLEX, 0.8, cv::Scalar(0,255,0), 2, cv::LINE_AA);
}
```

Below is an example of calling:

```
$ ./distance_estimation 0 DICT_4X4_1000 0 1 100
```

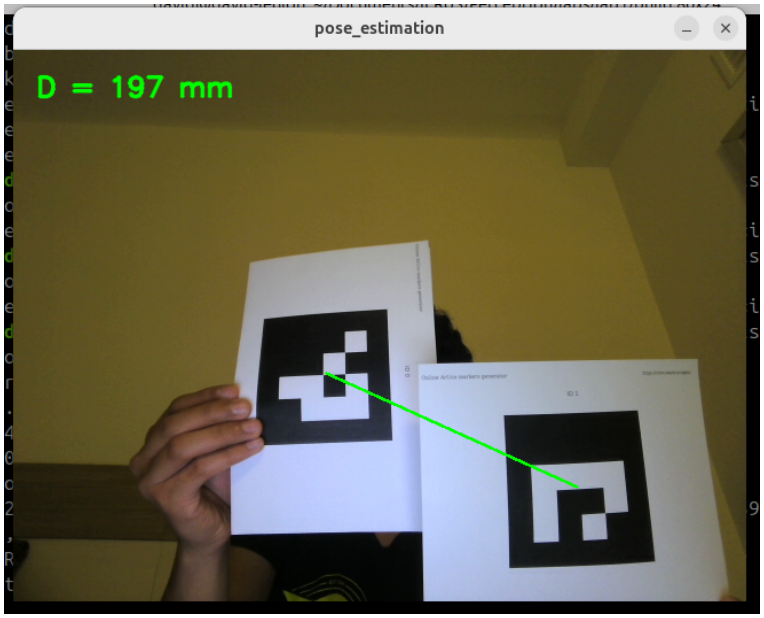

Figure 8: Example frame for calling /distance_estimation with the dictionary DICT_6X6_1000

Conclusion

In this laboratory session, we implemented a complete perception pipeline based on ArUco markers using OpenCV. Starting from the generation of individual markers and boards, we developed real-time detection and visualization tools capable of identifying markers and extracting their identifiers.

Using an ArUco board, we calibrated the camera and obtained its intrinsic parameters and distortion coefficients. These parameters were essential to perform accurate 3D pose estimation through the `solvePnP` method. Based on the estimated pose, we implemented augmented reality applications such as drawing reference axes and a virtual cube on top of a marker.

Furthermore, we estimated the relative pose between two markers and computed their relative translation using homogeneous transformations. Finally, we calculated the Euclidean distance between two markers and compared it with real measurements, obtaining consistent results with small acceptable errors.

Overall, this lab allowed us to better understand camera geometry, coordinate transformations, and the importance of calibration in 3D perception tasks. The practical implementation reinforced the theoretical concepts and demonstrated how fiducial markers can be effectively used in robotics applications such as localization, mapping, and augmented reality.

References

[1] fdcl-gwu, *ArUco Markers Repository*. Available at: `https://github.com/fdcl-gwu/aruco-markers` Accessed: 7 March 2026.